PYTHON MINI PROJECT REPORT

SHOP INVENTORY DATA ANALYSIS & DASHBOARD

Using Pandas, NumPy, Matplotlib and Scikit-learn

Submitted by: _____

Class / Roll No.: _____

Submitted to: _____

This report documents a runnable Python project that cleans shop inventory data, trains two machine-learning models, and creates a four-chart visual dashboard.

1. Abstract

The Shop Inventory Data Analysis & Dashboard project demonstrates how Python can be used to explore product sales information. It reads a CSV dataset, removes rows with missing values, trains a linear regression model to estimate units sold, trains a logistic regression model to classify an out-of-stock label, and generates four visualizations. A sample CSV file is included so the project can be executed immediately. The sample data is synthetic and should not be represented as real shop records.

2. Objectives

• Load and inspect shop data from a CSV file.
• Clean missing values before analysis.
• Predict Units_Sold using Price and Daily_Visitors with Linear Regression.
• Classify Out_of_Stock using Units_Sold with Logistic Regression.
• Display patterns using a scatter plot, histogram, line graph and correlation heatmap.

3. Tools and Technologies

Language: Python 3
Libraries: Pandas, NumPy, Matplotlib, Scikit-learn
Input: shop_data.csv
Outputs: terminal results and dashboard_output.png

4. Dataset Description

The supplied demonstration dataset contains 100 rows and five columns. Three rows contain a missing value to demonstrate cleaning; 97 complete rows remain after dropna(). Product_ID identifies each product. Price is the listed price, Daily_Visitors is a visitor-count feature, Units_Sold is the target for regression, and Out_of_Stock is a binary label (0 or 1) for classification. These records are generated example data, not collected business data.

5. Methodology

Step 1 — Data loading: Pandas reads shop_data.csv.
Step 2 — Cleaning: rows with missing values are removed using dropna().
Step 3 — Linear Regression: Price and Daily_Visitors are used to predict Units_Sold. The data is split into 80% training and 20% testing sets. Mean Squared Error (MSE) measures average squared prediction error; lower values are generally better for the same target scale.
Step 4 — Logistic Regression: Units_Sold is used to classify Out_of_Stock. Accuracy is the share of test predictions that match the labels.
Step 5 — Visualization: four charts are saved to dashboard_output.png.

6. Dashboard Components

1. **Scatter plot:** compares Price with Units_Sold to explore their relationship.
2. **Histogram:** shows how frequently different Units_Sold values occur.
3. **Line graph:** displays Units_Sold for the first 30 products in the cleaned dataset.
4. **Correlation heatmap:** displays pairwise correlations among Price, Daily_Visitors, Units_Sold and Out_of_Stock. Correlation indicates association and does not by itself prove causation.

7. Execution and Results

Item	Observed result
Original sample dataset	100 rows × 5 columns
Rows after removing missing values	97 rows × 5 columns
Linear Regression — Mean Squared Error	31.50
Logistic Regression — Test accuracy	65.00%
Dashboard	dashboard_output.png created
Execution status	Completed successfully

These are results from one run using the included synthetic dataset and a fixed random seed. Results may differ when the CSV is replaced with another dataset.

8. Conclusion

The project demonstrates a complete basic data-analysis workflow rather than code alone: it reads a data file, cleans the data, trains two machine-learning models, evaluates their test performance, and creates a visual dashboard. The supplied run completed and produced the expected output image. The classification accuracy is only a demonstration result and should not be treated as proof of reliable real-world stock prediction.

9. Limitations and Future Scope

• The included data is synthetic; use verified shop records for meaningful conclusions.
• The model uses a small set of features and may not capture seasonality, promotions, supply delays or stock-on-hand.
• More evaluation metrics and cross-validation could be added.
• A future version could include a simple GUI, interactive charts, database storage and stock-reorder alerts.

10. How to Run the Project

1. Install Python 3.
2. Install dependencies in a terminal: `pip install pandas numpy matplotlib scikit-learn`
3. Keep `shop_inventory_analysis.py` and `shop_data.csv` in the same folder.
4. Run: `python shop_inventory_analysis.py`
5. Check the terminal output and open `dashboard_output.png`.

Appendix: Files Provided

shop_inventory_analysis.py — runnable Python source code
shop_data.csv — synthetic sample input dataset
console_output.txt — captured console output
dashboard_output.png — generated four-chart dashboard
Project_Report.pdf — this report
Code_Print.pdf — formatted code for printing
Output_Print.pdf — formatted console output for printing